

# Event Media Platform

## Project Report

| Area         | Project detail                                                                                               |
|--------------|--------------------------------------------------------------------------------------------------------------|
| Project type | Full-stack web application for hosting, managing, searching, and sharing event media.                        |
| Frontend     | Next.js 16 App Router, React 19, Tailwind CSS, shadcn-style UI components, Zustand, Axios, Socket.IO client. |
| Backend      | Node.js, Express, TypeScript, Prisma ORM, PostgreSQL, Multer, Sharp, Socket.IO, JWT authentication.          |
| AI modules   | Python scripts for MobileNetV2-based image tagging and DeepFace-based face matching.                         |
| Storage      | Cloudinary integration with local uploads fallback.                                                          |
| Prepared on  | June 12, 2026                                                                                                |

### Abstract

The Event Media Platform is a media management system designed for events where large numbers of photos and videos must be uploaded, organized, searched, and shared. The application combines a modern web client, an API server, a relational database, automated image tagging, face-recognition based discovery, real-time notifications, and controlled media download features.

This report summarizes the project objectives, functional scope, technology stack, architecture, database design, implementation modules, deployment approach, and future enhancements.

# 1. Introduction and Objectives

Event photography commonly produces hundreds or thousands of files across multiple albums. Manual searching is slow, sharing is fragmented, and useful engagement data is often unavailable. This project addresses those issues by creating a single platform where events, media files, user interactions, and intelligent discovery features work together.

## Problem Statement

- Guests need a fast way to find photos that include them or match a specific event, tag, photographer, or category.
- Photographers need controlled upload, organization, and deletion workflows for event galleries.
- Organizers need visibility into media volume, likes, shares, and top-performing content.
- The platform must support both public galleries and private event access rules.

## Project Objectives

- 1 Build a responsive frontend for authentication, dashboards, event pages, gallery browsing, search, comments, favorites, and shared media links.
- 2 Implement a secure REST API with JWT-based authentication, role-aware access checks, and Prisma-backed persistence.
- 3 Automate media understanding through image tagging and face matching services.
- 4 Support image and video uploads with Cloudinary deployment readiness and local storage fallback.
- 5 Provide real-time notifications when users receive likes, comments, or tags.

## 2. Functional Requirements and Scope

### Major Features

| Feature          | Description                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| Authentication   | Registration, login, JWT token issuance, protected profile retrieval, and role assignment.                               |
| Event management | Create, view, update, delete, sort, and filter events with public/private visibility.                                    |
| Media management | Single and bulk upload, image optimization, video handling, deletion, favorites, likes, comments, and downloads.         |
| AI discovery     | Automatic hashtag generation from image content, file names, and event metadata; selfie upload and face-match discovery. |
| Sharing          | Generated shared media links, share-count tracking, and social sharing support in the frontend.                          |
| Notifications    | Persistent notification records plus Socket.IO delivery for likes, comments, and photo tags.                             |
| Analytics        | Dashboard summaries for event counts, media counts, engagement, and leading content.                                     |

### User Roles

- ADMIN: broad administrative access and moderation authority.
- PHOTOGRAPHER: event/media contribution workflows and private-event visibility.
- MEMBER: authenticated event participant with broader private gallery access than a public viewer.
- VIEWER: default registered role with basic browsing and interaction capabilities.

The implementation focuses on a working minimum production-style system: frontend flows, backend routes, persistence, AI services, upload handling, and deployment configuration are represented in the repository.

### 3. System Architecture

The platform follows a layered full-stack architecture. The Next.js client handles user interaction and calls the Express API. The API coordinates business rules, Prisma data access, file handling, AI subprocesses, and real-time Socket.IO messages. PostgreSQL stores structured records, while uploaded assets are stored in Cloudinary or the local uploads directory.

#### Architecture Flow

```
Browser / Next.js UI
  -> REST API + Socket.IO
  -> Controllers and Services
  -> Prisma ORM -> PostgreSQL
  -> Multer/Sharp -> Cloudinary or local uploads
  -> Python AI scripts -> MobileNetV2 tags / DeepFace matches
```

#### Technology Stack

| Layer     | Technology                                      | Purpose                                                                          |
|-----------|-------------------------------------------------|----------------------------------------------------------------------------------|
| Frontend  | Next.js 16, React 19, Tailwind CSS              | Dashboard UI, routing, media browsing, auth screens, responsive layout.          |
| State/API | Zustand, Axios, Socket.IO client                | Client state, API requests, and real-time event handling.                        |
| Backend   | Express 5, TypeScript, Socket.IO                | REST endpoints, request validation, authentication, notifications.               |
| Database  | Prisma 6, PostgreSQL                            | Typed ORM and relational storage for users, events, media, engagement, and tags. |
| AI        | Python, TensorFlow/Keras, MobileNetV2, DeepFace | Image classification tags and face-match search.                                 |
| Media     | Multer, Sharp, Cloudinary                       | Upload intake, image optimization, cloud storage, and local fallback.            |

## 4. Database and API Design

The Prisma schema models the platform around users, events, media, and engagement records. Relationships are explicit, and uniqueness constraints prevent duplicate likes, favorites, face matches, and user tags for the same media item.

### Core Data Model

| Entity      | Important fields                                  | Relationships                                                                                       |
|-------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| User        | id, name, email, password, role, selfieUrl        | Owns events/media; creates likes, comments, favorites, face matches, notifications, and media tags. |
| Event       | title, description, category, eventDate, isPublic | Created by one user and contains many media records.                                                |
| Media       | url, type, tags, shareCount                       | Belongs to an event and uploader; has likes, comments, favorites, tags, and face matches.           |
| Engagement  | Like, Comment, Favorite, Notification             | Captures social actions and user-facing activity updates.                                           |
| Recognition | FaceMatch, MediaTag                               | Connects users to matching or manually tagged media.                                                |

### API Modules

- Auth routes handle registration and login with bcrypt password hashing and JWT token creation.
- Event routes expose event CRUD operations with private-event visibility checks.
- Media routes cover upload, bulk upload, search, likes, comments, favorites, selfie recognition, manual tagging, sharing, downloads, and deletion.
- Notification routes support listing, unread counts, marking as read, and clearing notifications.
- Analytics routes support dashboard reporting for activity and engagement metrics.

Protected backend middleware decodes the JWT and attaches user identity and role to authenticated request objects.

## 5. Implementation Highlights

### Media Processing

- Images are optimized with Sharp to a maximum 1920 x 1080 size and JPEG quality settings before storage.
- Videos are recognized through file extensions and served with HTML5-compatible playback behavior in the frontend.
- Cloudinary upload is attempted first; if cloud configuration is unavailable, local upload paths remain usable.
- Image downloads receive a generated watermark containing club, event, and user-role context.

### AI Tagging and Face Recognition

The tagging pipeline calls a Python MobileNetV2 script, maps ImageNet predictions into event-friendly tags such as people, portrait, celebration, sports, dining, and outdoor, then supplements those tags with filename and event metadata keywords. The face matching pipeline stores a user's reference selfie and compares it against uploaded media using DeepFace, creating FaceMatch records for personal photo results.

### Frontend Experience

- Dashboard pages cover events, event details, event creation/editing, search, favorites, my photos, notifications, and user profiles.
- Reusable UI components include buttons, dialogs, sheets, inputs, cards, badges, tables, skeleton loading states, avatars, and dropdown menus.
- Gallery components provide likes, comments, favorites, media cards, upload controls, and shared media views.

## 6. Security, Deployment, Testing, and Conclusion

### Security Measures

- Passwords are hashed with bcrypt before being stored.
- JWTs carry user id and role information for protected route authorization.
- Private events are hidden from users who do not have sufficient role permissions.
- Media deletion checks ownership or ADMIN status before removing records and related engagement data.
- CORS is configured against the frontend origin supplied through environment variables.

### Deployment Model

| Component   | Deployment note                                                                                          |
|-------------|----------------------------------------------------------------------------------------------------------|
| Frontend    | Deploy the frontend directory to Vercel with NEXT_PUBLIC_API_URL pointing to the backend API.            |
| Backend     | Deploy the Express server to Render, Railway, Heroku, or similar Node-capable hosting.                   |
| Database    | Use a managed PostgreSQL service such as Supabase, Neon, AWS RDS, or another compatible provider.        |
| Environment | Configure DATABASE_URL, JWT_SECRET, FRONTEND_URL, and optional Cloudinary credentials.                   |
| AI runtime  | Install Python dependencies including DeepFace, TensorFlow/Keras, OpenCV, and NumPy on the backend host. |

### Conclusion and Future Scope

The Event Media Platform demonstrates a practical full-stack solution for event albums. It combines user management, event organization, media publishing, engagement tools, AI-assisted search, and deployment-ready infrastructure in a coherent application.

- Add automated API, frontend, and AI-service tests.
- Move AI jobs into a background queue for large upload and recognition workloads.
- Add admin moderation screens, audit logs, richer search filters, expiring share links, and observability.

References: repository README, package manifests, Prisma schema, backend controllers/services, frontend pages/components, and Python AI scripts.